

Web Development Lab

HTML | CSS | JavaScript | PHP

Part 1: Problem Summaries & Code Snippets

Each problem is summarised with its files, statement, approach, solution strategy, key functions, and the most important code snippet explaining the core logic.

Problem 1: Static HTML Pages

Three standalone HTML pages built using only HTML tags and inline attributes — no external CSS or JS.

Files	p1.html (Student Profile) p2.html (Event Details) p3.html (Company Page) bg.png, bg.jpg, dp.jpg, logo.png
Problem Statement	Create (a) a student profile page showing CGPA, hobbies, certifications and projects; (b) an event details page with background, image and links; (c) a company homepage with logo, marquee, description and services list.
Problem Approach	Each page is fully self-contained. Layout is controlled with HTML alignment attributes (align, bgcolor, background). No external files needed other than images. Focus is on mastering core HTML tags.
Solution Approach	p1: Center heading with <h1 align=center>, right-align photo with , use for coloured headings, for lists, <marquee> for scrolling text, <hr> as dividers. p2: Set body background with the background attribute, use <center> for centering, link to Google Maps with <a href target=_blank>. p3: Use background=bg.jpg on body, float logo right with , use text-align:justify inline on <p>.
Key Tags & Attributes	<html> <head> <body background bgcolor> <h1-h3> <p align> <a href target> <marquee behavior direction> <hr> <center>

p1.html — Core Code (Student Profile)

```

<marquee behavior="alternate" direction="right">
  <font color="green" size="4"><b>B.Tech CSE, Graphic Era University</b></font>
</marquee>
<h3><font color="Red">CGPA</font></h3>
<ul><font color="Orange">
  <li><b>Sem 1:</b> 9.70</li>
  <li><b>Sem 2:</b> 9.75</li>
</font></ul>
```

p2.html — Core Code (Event Page)

```
<body background="bg.png" style="text-shadow: 0.5px 1px 1px rgb(203,81,81);">
  <center>
    <h1>Work-Life Balance Seminar</h1>
    <p><b>Venue:</b>
      <a href="https://maps.app.goo.gl/..." target="_blank">Taj Yeshwantpur</a>
```

```

    </p>
    
  </center>
</body>

```

p3.html — Core Code (Company Page)

```

<body background="bg.jpg" style="background-size:cover;">
  <h1>Creadeau Solutions
    
  </h1>
  <marquee direction="right">
    <font color="green" size="4">Welcome to Creadeau Solutions</font>
  </marquee>
  <p align="Justify">Credeau is a fintech company...</p>
</body>

```

Problem 2: HTML Lists & Tables

Demonstrate all three HTML list types (unordered, ordered, definition) with nested lists, and build a complex multi-span timetable.

Files	p1.html (CS Department page with lists) p2.html (Weekly timetable)
Problem Statement	p1: Show department facilities (ul), semester subjects (ol), course descriptions (dl), and subjects with sub-units (nested ul). p2: Render a Monday–Friday class timetable with proper colspan/rowspan for shared slots.
Problem Approach	p1 stacks the three list types vertically. Nested inside creates the sub-topic tree. p2 is a table where rowspan=5 spans LUNCH across all five days and colspan=2 merges consecutive lab slots.
Solution Approach	p1: Use for bullets, for numbered list, <dl><dt><dd> for definitions. For nested list, add a second as a child . p2: Set border/cellpadding/cellspacing on <table>. Color alternate rows with different bgcolor. Use colspan/rowspan carefully so total column widths stay consistent.
Key Tags	<dl> <dt> <dd> <table border cellpadding cellspacing> <tr bgcolor> <th> <td colspan rowspan> nested inside

p1.html — All Three List Types

```

<h3>Facilities</h3>
<ul>                                <!-- Unordered / bullet list -->
  <li>Computer Lab</li>
  <li>Library</li>
</ul>
<h3>Subjects</h3>
<ol>                                <!-- Ordered / numbered list -->
  <li>Programming in C</li>
  <li>Python Programming</li>
</ol>
<h3>Courses</h3>

```

```

<dl>                                <!-- Definition list -->
  <dt>B.Tech</dt>
  <dd>4-year undergraduate engineering program</dd>
  <dt>M.Tech</dt>
  <dd>2-year postgraduate engineering program</dd>
</dl>
<!-- Nested list: ul inside li -->
<ul>
  <li>Data Structures in C
    <ul>                                <!-- Sub-list -->
      <li>Arrays</li>
      <li>Stacks</li>
    </ul>
  </li>
</ul>

```

p2.html — Table with rowspan & colspan

```

<table border="2" align="center" cellpadding="6" cellspacing="0">
  <tr bgcolor="#751400" style="color:white;">
    <th bgcolor="white" style="color:black;">Day</th>
    <th>8:00-8:55</th> <th>9:00-9:55</th> ...
  </tr>
  <tr bgcolor="#E97451">
    <th bgcolor="#751400" style="color:white;">Monday</th>
    <td>TCS-611</td>
    <td>TCS-693</td>
    <td rowspan="5" align="center"><b>LUNCH</b></td> <!-- spans 5 rows -->
    <td colspan="2">PCS-601 Lab</td> <!-- spans 2 cols -->
  </tr>
  <!-- Repeat for Tuesday-Friday with matching spans -->
</table>

```

Problem 3: HTML Forms with Validation

Two forms demonstrating HTML5 client-side validation — no JavaScript needed.

Files	p1.html (Student Course Registration) p2.html (Job Application Form)
Problem Statement	p1: Collect student name (alpha only), university email, roll number (pattern-validated), course (dropdown), age (18-30), gender (radio). p2: Collect full name, email, mobile (Indian format), experience (0-30), job role, and a cover letter textarea.
Problem Approach	Use HTML5 constraint attributes on <input> elements. The browser's built-in validation engine handles all checks — no JS required. Use internal <style> for layout. Flexbox places the form and logo side by side in p1.
Solution Approach	pattern='[a-zA-Z][a-zA-Z]+' enforces name format. pattern='[A-Z]{2}[0-9]{2}[A-Z]{3}' validates roll number. type='email' auto-validates email format. type='number' with min/max handles age/experience. pattern='[6-9][0-9]{9}' validates Indian mobile numbers. title attribute shows a tooltip when validation fails.

Key Attributes

required | pattern | minlength maxlength | min max | title |
 placeholder | type=email/number/radio/text | <select required> |
 <textarea minlength maxlength rows cols>

p1.html — Form with Flexbox Layout & Pattern Validation

```
<form>
  <div style="display:flex;"                <!-- Side-by-side layout -->
    <div class="m2">                      <!-- Left: form fields -->
      Name: <input type="text" name="studentName"
        required pattern="[a-zA-Z][a-zA-Z ]+"
        title="Enter valid name" minlength="3" maxlength="30">

      Email: <input type="email" name="email" required>

      Roll No: <input type="text" name="roll"
        pattern="[A-Z]{2}[0-9]{2}[A-Z]{3}"
        title="Format: 2 uppercase, 2 digits, 3 uppercase">

      Age: <input type="number" name="age" min="18" max="30">

      Gender:
        <input type="radio" name="gender" value="male"> Male
        <input type="radio" name="gender" value="female"> Female
    </div>
    <div class="m5">                      <!-- Right: logo image -->
      
    </div>
  </div>
  <input type="submit"> <input type="reset">
</form>
```

p2.html — Mobile Validation & Textarea

```
Mobile: <input type="text" name="mobilenumber"
  maxlength="10"
  pattern="[6-9][0-9]{9}"
  title="Must start 6-9, 10 digits total">

Experience: <input type="number" name="experience"
  min="0" max="30" step="1">

<textarea name="coverletter"
  minlength="100" maxlength="1000"
  rows="10" cols="30"
  placeholder="Cover Letter"></textarea>
```

Problem 4: Frames-Based Website

A multi-pane website using HTML framesets — header, navigation menu, and content all in separate files.

| | |
|--------------------------|---|
| Files | p1.html (frameset container) p1a.html (header frame) p1b.html (menu frame) p1c.html (content frame) |
| Problem Statement | Split the browser window into a header row (top 20%) and a lower section divided into a menu column (15%) and content area (85%). Menu links must load pages into the content frame without refreshing the header or menu. |
| Problem Approach | Outer <frameset rows='20%,80%'> handles vertical split. Inner <frameset cols='15%,85%'> handles horizontal split of the lower section. Each <frame> tag has a name attribute used as a target by menu links. |
| Solution Approach | p1b (menu) links use target='contentFrame' to load pages into the content area only. p1c (content) uses named anchors for in-page navigation. noresize on header/menu frames prevents accidental resizing. |
| Key Tags | <frameset rows cols> <frame src name noresize> <a href target='frameName'> bgcolor on body list-style-type:none to remove bullet points from menu |

p1.html — Nested Frameset Structure

```
<html>
<body>
  <frameset rows="20%,80%">                <!-- outer: top/bottom split -->
    <frame name="headerFrame" src="p1a.html" noresize>
    <frameset cols="15%,85%">              <!-- inner: left/right split -->
      <frame name="menuFrame" src="p1b.html" noresize>
      <frame name="contentFrame" src="p1c.html">
    </frameset>
  </frameset>
</body>
</html>
```

p1b.html — Menu Links Targeting Content Frame

```
<ul style="list-style-type:none; padding-left:10px;">
  <!-- target="contentFrame" loads page there, not in whole window -->
  <li><a href="content.html#about" target="contentFrame">About Us</a></li>
  <li><a href="content.html#products" target="contentFrame">Products</a></li>
  <li><a href="content.html#announcements" target="contentFrame">Announcements</a></li>
  <li><a href="content.html#contact" target="contentFrame">Contact Us</a></li>
</ul>
```

p1c.html — Named Anchors for In-Page Navigation

```
<a name="about"></a>                <!-- anchor: scrolls here when #about clicked -->
<h2>About Us</h2>
<p>Creative Solutions is a leading company...</p>
<hr>
<a name="products"></a>
<h2>Our Products</h2>
<ul><li>Web Development Services</li> ...</ul>
```

Problem 5: Image Map (Hospital Navigation)

A clickable hospital map where three differently-shaped regions navigate to separate department pages, all styled with shared external CSS.

Files	p1.html (image + map) p1a.html (OPD) p1b.html (Emergency) p1c.html (Diagnostic Lab) s.css (shared stylesheet) bg.png (hospital map image)
Problem Statement	Display a hospital map image. Clicking the OPD zone opens the OPD page, clicking the Emergency circle opens Emergency, clicking the hexagonal Diagnostic Lab area opens that page.
Problem Approach	The attribute links the image to a <map> element. Three <area> children define the clickable zones with their geometric shapes and pixel coordinates determined from the image.
Solution Approach	OPD: shape='rect' with coords='x1,y1,x2,y2' (top-left to bottom-right corners). Emergency: shape='circle' with coords='cx,cy,radius'. Diagnostic Lab: shape='poly' with coords='x1,y1,x2,y2,...' listing polygon vertices. s.css defines shared styles for all department pages.
Key Tags & CSS	 <map name='name'> <area shape='rect/circle/poly' coords href alt> <link rel='stylesheet' href='s.css'> CSS: font-family, background-color, color, border-bottom on h2, line-height

p1.html — Image Map with Three Shape Types

```


<map name="hos">
  <!-- Rectangle: coords = x1,y1 (top-left), x2,y2 (bottom-right) -->
  <area shape="rect"    coords="250,500,430,560"                href="p1a.html" alt="OPD">

  <!-- Circle: coords = centerX,centerY,radius -->
  <area shape="circle"  coords="770,520,60"                    href="p1b.html"
alt="Emergency">

  <!-- Polygon: coords = x1,y1,x2,y2,x3,y3,... (each vertex) -->
  <area shape="poly"    coords="1180,490,1240,470,1300,490,
                                1320,540,1300,590,1240,610,
                                1180,590,1160,540"                href="p1c.html"
alt="Diagnostic Lab">
</map>
```

s.css — Shared Stylesheet for Department Pages

```
body {
  font-family: Arial, sans-serif;
  background-color: #f5f7fa;
  padding: 20px;
  line-height: 1.6;
}
h1 { text-align: center; color: #0b3c5d; }
h2 {
  color: #1f6f8b;
  border-bottom: 2px solid #dbe9f4; /* Underline effect */
  padding-bottom: 5px;
}
hr { border:none; height:2px; background-color:#1f6f8b; }
```

Problem 6: Feedback Form with Confirmation Page

A student feedback form that navigates to a success confirmation page on submit — using external CSS and a flex header.

Files	p1.html (feedback form) p1a.html (success confirmation) s.css (external stylesheet)
Problem Statement	Build a university feedback form for name, email, course and comments. On submission, navigate to a separate page showing a green checkmark and a 'Form successfully filled' message.
Problem Approach	Set the form's action='p1a.html' so the browser navigates there on submit. A flex row in the header places the college name and logo side by side. External s.css handles body and heading colours; internal <style> handles form-specific styling.
Solution Approach	p1.html: The .header-container div uses display:flex with align-items:center and justify-content:center to place name and logo side by side. The form uses a white .form-container div. p1a.html: A centered .message-box div uses margin:0 auto. The checkmark is an HTML entity ✔ styled at font-size:50px.
Key Techniques	External <link rel='stylesheet'> Internal <style> for page-specific rules display:flex + align-items + justify-content form action='page.html' HTML entity ✔ (checkmark) margin:0 auto (centering)

p1.html — Flex Header + Form

```
<link rel="stylesheet" href="s.css">    <!-- external CSS -->

<div class="header-container">          <!-- flex row -->
  <div>
    <p class="college-name">Graphic Era (Deemed To Be) University</p>
  </div>
  <div>
    
  </div>
</div>

<!-- form action points to confirmation page, no JS needed -->
<form action="p1a.html">
  <input type="text" name="studentName" required>
  <input type="email" name="email" required>
  <select name="course" required>
    <option value="">-- Select Course --</option>
    <option value="B.Tech">B.Tech</option>
  </select>
  <textarea name="feedback" rows="4" required></textarea>
  <input type="submit" value="Submit">
</form>
```

p1a.html — Confirmation Page with Checkmark Entity

```

<style>
  .message-box {
    width: 400px;
    margin: 0 auto;          /* centers the box horizontally */
    padding: 40px;
    border: 2px solid black;
    text-align: center;
  }
  h1 { color: green; }
</style>

<div class="message-box">
  <div style="font-size:50px;">&#10004;</div>  <!-- ✓ checkmark entity -->
  <h1>Form successfully filled</h1>
  <h3>Thank you for filling the form!!</h3>
  <a href="p1.html">Go back to form</a>
</div>

```

Problem 7: JavaScript Calculator

A single-file calculator using JavaScript onclick handlers and eval() to evaluate expressions.

Files	p1.html (all-in-one: HTML + CSS + JavaScript)
Problem Statement	Build a working calculator with digit buttons (0-9), operator buttons (+, -, *, /, **), an AC (clear) button, and an = button. Show the current expression in a disabled text input.
Problem Approach	Each button has an onclick='functionName()' attribute. Every digit and operator function appends its character to the display input's value. The equal function reads the complete expression string and evaluates it with eval().
Solution Approach	All digit functions follow the same pattern: getElementById('display').value += 'digit'. Operator functions do the same. clea() resets to empty string. equal() wraps eval() in try-catch so invalid expressions show 'Error' instead of crashing. button: hover CSS gives visual feedback.
Key JS Concepts	document.getElementById(id) .value (read/write input content) += (string append) eval(string) — executes expression try {} catch(e) {} — error handling onclick HTML attribute

p1.html — HTML Button Layout

```

<div class="m2">
  <button onclick="one()">1</button>
  <button onclick="two()">2</button>
  <button onclick="three()">3</button>
  <button onclick="div()">/</button>
  <!-- ... rows for 4-6-*, 7-9--, 0-***= -->
  <button onclick="clea()">AC</button>
  <input type="text" id="display" disabled
    style="width:140px; height:40px; font-size:20px;">

```

</div>

p1.html — Core JavaScript Logic

```
// Every digit button follows the same pattern:
function one() { document.getElementById("display").value += "1"; }
function two() { document.getElementById("display").value += "2"; }
// ... same for three() through nine() and zero()

// Operators append their symbol:
function plus() { document.getElementById("display").value += "+"; }
function minus() { document.getElementById("display").value += "-"; }
function mul() { document.getElementById("display").value += "*"; }
function div() { document.getElementById("display").value += "/"; }
function power() { document.getElementById("display").value += "**"; }

// Clear: reset display to empty string
function clea() { document.getElementById("display").value = ""; }

// Equal: evaluate the expression string
function equal() {
  var display = document.getElementById("display").value;
  try {
    var result = eval(display); // e.g. eval("3+5*2") => 13
    document.getElementById("display").value = result;
  } catch(error) {
    document.getElementById("display").value = "Error";
  }
}
```

Problem 8: Exam Registration & Dynamic Admit Card

Three JS dialog functions and a dynamically generated admit card in a new browser window.

Files	p1.html (all-in-one: HTML + CSS + JavaScript) dp.jpg (student photo)
Problem Statement	Three buttons: (1) show exam instructions via alert, (2) register via prompt/confirm dialogs capturing name and roll number, (3) generate a printable admit card in a new window — only if registered.
Problem Approach	Alert() uses alert(). Prompt() chains two prompt() calls then a confirm(). Open() guards with an if-check, calls window.open() to create a blank window, then uses win.document.write() with a template literal to inject full HTML including student details and an exam schedule table.
Solution Approach	Variables name and ro are declared at script scope so all three functions share them. Prompt() stores or clears them based on user input. Open() checks if they are non-empty before proceeding. Template literals (backtick strings) allow \${name} and \${ro} to be embedded directly inside the HTML string written to the new window. window.print() and window.close() are called from buttons inside the new window.
Key JS Concepts	alert(msg) prompt(msg) — returns string or null confirm(msg) — returns true/false window.open(url, target, features)

```
win.document.write(html) | win.document.close() | Template literals
`${variable}` | window.print() | window.close()
```

p1.html — Shared Variables & Alert Function

```
let win;           // reference to popup window
let name = "";     // stored after registration
let ro  = "";      // roll number

function Alert() {
    alert("Important Exam Instructions\n\n" +
        "1. No electronic devices allowed\n" +
        "2. Bring your ID card\n" +
        "3. Reach 30 minutes early");
}
```

p1.html — Prompt + Confirm Registration

```
function Prompt() {
    name = prompt("Enter your name:");
    ro  = prompt("Enter your roll number:");
    if (!name || !ro) {
        alert("Registration cancelled or incomplete.");
        return;
    }
    let confirmRegister = confirm("Are you sure you want to register?");
    if (confirmRegister) {
        alert("Successfully registered!");
    } else {
        name = ""; ro = ""; // clear if cancelled
        alert("Registration cancelled.");
    }
}
```

p1.html — Dynamic Admit Card in New Window

```
function Open() {
    if (!name || !ro) { alert("Please register first!"); return; }

    // Open blank popup window
    win = window.open("", "_blank", "width=800,height=800");

    // Write full HTML using template literal - ${name} inserts value
    win.document.write(`
        <html><body>
            <h1>Graphic Era University</h1>
            
            <p><b>Name:</b> ${name}</p>
            <p><b>Roll No:</b> ${ro}</p>
            <table border="1">
                <tr><th>Subject</th><th>Date</th><th>Venue</th></tr>
                <tr><td>Web Dev Lab</td><td>27-02-2026</td><td>Lab 2</td></tr>
            </table>
            <button onclick="window.print()">Print</button>
            <button onclick="window.close()">Close</button>
        </body></html>
    `);
    win.document.close(); // finalize the document
}
```

```
}
```

Problem 9: College Homepage with Background Colour Cycler

A college info page with a button that cycles through 10 colours using an array and `addEventListener`.

Files	p1.html (all-in-one: HTML + CSS + JavaScript)
Problem Statement	Display a college homepage with facts, campus images, and a button. Each click of the button changes the background colour to the next colour in a predefined list, wrapping back to start after the last one.
Problem Approach	Declare a const array of 10 hex colour strings and a let counter starting at 0. Use <code>document.getElementById</code> to get the button and <code>addEventListener('click')</code> to attach a handler. Inside the handler, set <code>body.style.backgroundColor</code> to <code>colors[i]</code> , then advance <code>i</code> using modulo.
Solution Approach	The modulo operation <code>i = (i+1) % colors.length</code> ensures <code>i</code> wraps from 9 back to 0 after the last colour. The circular cycling works without any if-statement. The colours are all blue-toned shades matching the college theme.
Key JS Concepts	<code>const array = [...]</code> <code>let counter = 0</code> <code>document.getElementById()</code> <code>.addEventListener('click', fn)</code> <code>document.body.style.backgroundColor</code> modulo operator: <code>i = (i+1) % array.length</code>

p1.html — Colour Cycling with `addEventListener` & Modulo

```
<button id="btn">Change Background Color</button>

<script>
  const button = document.getElementById("btn");

  // Array of 10 blue-toned hex colours
  const colors = [
    "#b0c4de", "#add8e6", "#87ceeb", "#4682b4", "#5f9ea0",
    "#6495ed", "#1e90ff", "#4169e1", "#6a5acd", "#778899"
  ];

  let i = 0;    // current colour index

  button.addEventListener("click", function() {
    document.body.style.backgroundColor = colors[i];
    i = (i + 1) % colors.length;  // wrap: 9->0, 0->1, 1->2...
  });
</script>
```

Problem 10: Dynamic Calendar Generator

A month+year dropdown picker that dynamically builds a calendar table using JavaScript's Date object and DOM creation.

Files	p1.html (all-in-one: HTML + CSS + JavaScript)
Problem Statement	Show two dropdowns — month and year. When both are selected, render the correct calendar grid with day-of-week headers and dates aligned to the right starting day.
Problem Approach	Two event listeners ('change') on the selects call renderCalendar(). The function uses new Date(year, month, 1).getDay() to find what weekday the 1st falls on (0=Sun, 6=Sat). It then uses new Date(year, month+1, 0).getDate() to find the total days in the month. A nested loop creates table rows and cells dynamically.
Solution Approach	renderCalendar clears tbody.innerHTML, creates a header row with day abbreviations, then loops up to 6 rows x 7 columns. For row 0, the first fd cells are empty (the offset). After the offset, date numbers fill in order. When date > daysInMonth, remaining cells are empty. Rows where all cells are empty are skipped with an emptyRow flag.
Key JS Concepts	new Date(y,m,1).getDay() — weekday of 1st new Date(y,m+1,0).getDate() — days in month trick document.createElement(tag) element.appendChild(child) element.textContent = text parseInt() isNaN() innerHTML = " to clear

p1.html — Full renderCalendar() Function

```
const ms = document.getElementById("ms"); // month select
const ys = document.getElementById("ys"); // year select
const cb = document.getElementById("cb"); // tbody
const mh = document.getElementById("mh"); // header cell
const days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
const months= ["January", "February", "March", "April", "May", "June",
               "July", "August", "September", "October", "November", "December"];

function renderCalendar() {
  const month = parseInt(ms.value);
  const year = parseInt(ys.value);
  if (isNaN(month) || isNaN(year)) return; // nothing selected yet

  mh.textContent = months[month] + " " + year; // header e.g. "May 2026"
  cb.innerHTML = ""; // clear previous grid

  // --- Header row: Sun Mon Tue ... Sat ---
  const hr = document.createElement("tr");
  days.forEach(d => {
    const th = document.createElement("th");
    th.textContent = d;
    hr.appendChild(th);
  });
  cb.appendChild(hr);

  // --- Key calculations ---
  const fd = new Date(year, month, 1).getDay(); // e.g. 3 = Wednesday
  const dm = new Date(year, month+1, 0).getDate(); // e.g. 31 for May
  // ^^ month+1, day 0 = last day of current month
```

```

let date = 1;
for (let row = 0; row < 6; row++) {
  const tr = document.createElement("tr");
  let emptyRow = true;
  for (let col = 0; col < 7; col++) {
    const td = document.createElement("td");
    if (row === 0 && col < fd) {
      td.textContent = ""; // blank cells before 1st day
    } else if (date > dm) {
      td.textContent = ""; // blank cells after last day
    } else {
      td.textContent = date;
      date++;
      emptyRow = false;
    }
    tr.appendChild(td);
  }
  if (!emptyRow) cb.appendChild(tr); // skip fully-empty 6th row
}

ms.addEventListener("change", renderCalendar);
ys.addEventListener("change", renderCalendar);

```

Problem 11: Stopwatch

A mm:ss:cs stopwatch with Start, Pause, and Reset using setInterval with cascading counter logic.

Files	p1.html (all-in-one: HTML + CSS + JavaScript)
Problem Statement	Build a stopwatch displaying minutes, seconds, and centiseconds (hundredths of a second). Start begins counting, Pause stops it without resetting, Reset zeros everything.
Problem Approach	Three integer counters (ms, s, m) track centiseconds, seconds, minutes. setInterval fires every 10ms incrementing ms. When ms hits 100 it resets to 0 and increments s. When s hits 60 it resets to 0 and increments m. A timer variable holds the interval ID; null means stopped.
Solution Approach	Start(): Guard against double-start with if(timer!==null)return. Call setInterval(callback, 10). Callback increments ms and cascades. Update three elements with zero-padded strings using ternary. Pause(): clearInterval(timer); timer=null. Reset(): clearInterval, null timer, zero all counters, reset all span texts.
Key JS Concepts	setInterval(fn, ms) — repeating timer clearInterval(id) — stop timer let timer = null as stopped state sentinel element.innerText = value Ternary zero-padding: n<10 ? '0'+n : n Arrow function callback: ()=>{}

p1.html — Stopwatch Display HTML

```
<div id="stopwatch" class="mm">
```

```
<font size="10px">
  <span id="m">00</span> :
  <span id="s">00</span> :
  <span id="mi">00</span>
</font>
</div>
<button onclick="Start()">Start</button>
<button onclick="Pause()">Pause</button>
<button onclick="Reset()">Reset</button>
```

p1.html — Core Stopwatch JavaScript

```
let ms=0, s=0, m=0;    // counters
let timer = null;      // null = stopped, ID = running

const ax = document.getElementById("mi"); // centiseconds span
const bx = document.getElementById("s");  // seconds span
const cx = document.getElementById("m");  // minutes span

function Start() {
  if (timer !== null) return; // already running - do nothing
  timer = setInterval(() => {
    ms++;
    if (ms === 100) { ms = 0; s++; } // 100 centiseconds = 1 second
    if (s === 60) { s = 0; m++; } // 60 seconds = 1 minute
    // Zero-pad: if ms<10 show "0"+ms else show ms
    ax.innerText = ms < 10 ? "0" + ms : ms;
    bx.innerText = s < 10 ? "0" + s : s;
    cx.innerText = m < 10 ? "0" + m : m;
  }, 10); // fires every 10ms
}

function Pause() {
  clearInterval(timer); // stop the interval
  timer = null;         // mark as stopped
}

function Reset() {
  clearInterval(timer);
  timer = null;
  ms = 0; s = 0; m = 0;
  ax.innerText = "00"; bx.innerText = "00"; cx.innerText = "00";
}
```

Part 1 (continued): PHP Problems 12–19

Problem 12: PHP Basics — 5 Mini Programs

Five foundational PHP programs covering: echo, even/odd, largest of three, average & smallest, and value swapping.

Files	p1.php (echo) p2.php (even/odd) p3.php (largest of 3) p4.php (average & smallest) p5.php (swap)
Problem Statement	p1: Print 'Welcome to PHP'. p2: Accept a number and say Even or Odd. p3: Accept three numbers and find the largest. p4: Accept three numbers, calculate average and find the smallest. p5: Accept two numbers and swap them using a temporary variable.
Problem Approach	Each file uses a self-submitting form (action="") with method='post'. PHP processes when \$_SERVER['REQUEST_METHOD']=='POST'. Values are read from \$_POST. Results are echoed below the form on the same page.
Solution Approach	p2: $\$n \% 2 == 0$ tests even. p3/p4: if-elseif-else chain compares three values. p4: Sum all three and divide by 3 for average. p5: The named submit button (name='swap' value='swap') is detected via \$_POST['swap'] to trigger the swap logic. Temp variable pattern: $\$temp = \$a; \$a = \$b; \$b = \$temp$.
Key PHP Concepts	<?php ?> tags echo \$variable \$_POST['key'] \$_SERVER['REQUEST_METHOD'] if / elseif / else % modulo +, -, *, / temp variable swap named submit button detection

p1.php — Hello PHP (Basic Echo)

```
<?php
    echo "Welcome to PHP";
?>
```

p2.php — Even or Odd

```
<form method="post">
    <input type="number" name="number">
    <button type="submit">Check</button>
</form>

<?php
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        $n = $_POST["number"];
        if ($n % 2 == 0)
            echo "$n is Even";
        else
            echo "$n is Odd";
    }
?>
```

p3.php — Largest of Three Numbers

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $a = $_POST["num1"];
    $b = $_POST["num2"];
    $c = $_POST["num3"];
    if ($a > $b && $a > $c)
        echo "Largest: $a";
    elseif ($b > $a && $b > $c)
        echo "Largest: $b";
    else
        echo "Largest: $c";
}
?>
```

p4.php — Average & Smallest

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $a = $_POST["num1"]; $b = $_POST["num2"]; $c = $_POST["num3"];
    $avg = ($a + $b + $c) / 3;
    echo "Average: $avg";
    if ($a < $b && $a < $c)      echo " | Smallest: $a";
    elseif ($b < $a && $b < $c)  echo " | Smallest: $b";
    else                        echo " | Smallest: $c";
}
?>
```

p5.php — Swap Two Values (Temp Variable)

```
<!-- Named submit button: name="swap" value="swap" -->
<button type="submit" name="swap" value="swap">Swap</button>

<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $a = $_POST["num1"];
    $b = $_POST["num2"];
    if ($_POST["swap"] == "swap") {
        echo "Before: a=$a, b=$b";
        $temp = $a;    // save a
        $a     = $b;    // overwrite a with b
        $b     = $temp; // put saved a into b
        echo "<br>After:  a=$a, b=$b";
    }
}
?>
```

Problem 13: PHP Calculator with Switch

A two-number calculator using four operator submit buttons and a switch statement.

Files
p1.php (single file — form + PHP)

Problem Statement	Two number inputs and four buttons (+, -, *, /). Clicking an operator button submits the form with that operator's value, and PHP computes and displays the result using a switch statement.
Problem Approach	All four buttons are type='submit' with name='operator' but different value attributes (+, -, *, /). PHP reads which button was clicked via \$_POST['operator'] and uses switch to branch to the correct arithmetic.
Solution Approach	Cast inputs to float with (float) to handle decimal numbers. switch(\$operator) has four cases and a guard inside the division case checking \$b!=0 to prevent division by zero. Result is displayed in a styled <div>.
Key PHP Concepts	switch / case / break / default (float) type casting Multiple submit buttons with same name, different value Division by zero guard echo with HTML string concatenation String concatenation with dot (.)

p1.php — Multiple Operator Submit Buttons

```
<!-- All 4 buttons share name="operator", differ only in value -->
<button type="submit" name="operator" value="+">>+</button>
<button type="submit" name="operator" value="-">>-</button>
<button type="submit" name="operator" value="*">*</button>
<button type="submit" name="operator" value="/">/</button>
```

p1.php — Switch Statement Calculator

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $a = (float)$_POST["num1"];    // cast to float for decimals
    $b = (float)$_POST["num2"];
    $operator = $_POST["operator"];
    $result = "";

    switch ($operator) {
        case "+": $result = $a + $b; break;
        case "-": $result = $a - $b; break;
        case "*": $result = $a * $b; break;
        case "/":
            if ($b != 0) $result = $a / $b;
            else $result = "Error: Division by zero";
            break;
    }
    echo "<div><strong>$a $operator $b = </strong>$result</div>";
}
?>
```

Problem 14: PHP Type Functions — gettype, settype, isset, unset

Three programs demonstrating PHP's type-inspection and variable-management built-in functions.

Files	p1.php (gettype + settype) p2.php (isset demo) p3.php (unset demo)
--------------	--

Problem Statement	p1: Show a variable's type at each step as it is converted from string to integer to boolean. p2: Form with input — use <code>isset()</code> to detect if the form was submitted and if the input is non-empty. p3: Set a variable, use <code>isset()</code> to confirm it exists, <code>unset()</code> to destroy it, then confirm it's gone.
Problem Approach	p1 runs pure PHP on page load (no form needed) — builds a <code>\$log</code> array of strings, iterates it with <code>foreach</code> in HTML. p2 uses a form; <code>isset()</code> checks two conditions. p3 uses <code>unset()</code> and demonstrates the variable is truly gone with a second <code>isset()</code> check.
Solution Approach	p1: <code>gettype(\$var)</code> returns the type as a string. <code>settype(\$var,'integer')</code> changes type in-place. A <code>\$log</code> array captures the state after each step and <code>foreach</code> renders each entry in a styled <code><div></code> . p2: <code>isset(\$_POST['submit_btn']) AND \$_POST['user_input']!="</code> together confirm a valid submission. p3: The pattern shows before-and-after clearly.
Key PHP Functions	<code>gettype(\$var)</code> — returns 'string','integer','boolean',etc <code>settype(\$var,'type')</code> — converts type in-place <code>isset(\$var)</code> — true if set and not null <code>unset(\$var)</code> — destroys variable <code>htmlspecialchars()</code> — safe output <code>foreach(\$arr as \$item)</code> <code>\$log[] = 'value'</code> — append to array

p1.php — `gettype()` and `settype()` Chain

```
<?php
    $var = "100";           // starts as string
    $log = [];              // empty log array

    $log[] = "Value: $var | Type: " . gettype($var);    // "string"

    settype($var, "integer");
    $log[] = "Value: $var | Type: " . gettype($var);    // "integer"

    settype($var, "boolean");
    $log[] = "Value: " . ($var ? "true":"false") . " | Type: " . gettype($var);    //
"boolean"
?>
<html><body>
    <?php foreach ($log as $message): ?>
        <div class="result"><?php echo $message; ?></div>
    <?php endforeach; ?>
</body></html>
```

p2.php — `isset()` to Detect Form Submission

```
<?php
    $message = "";
    if (isset($_POST["submit_btn"])) {                // was submit clicked?
        if (isset($_POST["user_input"]) && $_POST["user_input"] != "") {
            $safe = htmlspecialchars($_POST["user_input"]); // prevent XSS
            $message = "Variable is set! Value: <strong>$safe</strong>";
        } else {
            $message = "Form submitted but input is empty.";
        }
    }
?>
```

p3.php — unset() Variable Destruction

```
<?php
    $my_secret = "PHP is awesome!";
    $log = [];

    if (isset($my_secret))
        $log[] = "BEFORE unset(): set. Value = $my_secret";

    unset($my_secret);    // destroy the variable

    if (isset($my_secret))
        $log[] = "AFTER unset(): still set (should not appear)";
    else
        $log[] = "AFTER unset(): variable is GONE.";
?>
```

Problem 15: PHP Session-Based Login System

A single-file login system using PHP sessions — showing login form, welcome dashboard, and logout in one page.

Files	p1.php (single file — handles all three states: login form, dashboard, logout)
Problem Statement	Build a login page where username='admin' and password='password' starts a session and shows a welcome dashboard. A logout link destroys the session and returns to the login form.
Problem Approach	The page has three logical states handled by PHP at the top before any HTML: (1) Logout action — destroy session and redirect. (2) Login form submitted — validate credentials, set session, redirect. (3) Normal load — render login form or dashboard based on session existence.
Solution Approach	session_start() is called first, always. Logout: isset(\$_GET['action'])=='logout' triggers session_unset() then session_destroy() then header redirect. Login: if credentials match, set \$_SESSION['logged_in']=true and \$_SESSION['user']. HTML uses if(isset(\$_SESSION['logged_in'])) to switch between dashboard and login form views.
Key PHP Concepts	session_start() — must be FIRST \$_SESSION['key'] = value session_unset() — clears data session_destroy() — ends session header('Location: url') — redirect exit() — stop script after redirect \$_GET['action'] isset() htmlspecialchars()

p1.php — Full Session Flow (top of file, before HTML)

```
<?php
session_start();    // MUST be first — before any HTML or whitespace
$error = "";

// STATE 1: Logout — destroy session and redirect
if (isset($_GET["action"]) && $_GET["action"] == "logout") {
    session_unset();    // remove all session variables
```

```

    session_destroy(); // end the session
    header("Location: " . $_SERVER["PHP_SELF"]);
    exit();             // always exit after header redirect
}

// STATE 2: Login form submitted
if (isset($_POST["login"])) {
    $username = $_POST["username"];
    $password = $_POST["password"];
    if ($username === "admin" && $password === "password") {
        $_SESSION["logged_in"] = true;
        $_SESSION["user"]      = $username;
        header("Location: " . $_SERVER["PHP_SELF"]);
        exit();
    } else {
        $error = "Invalid username or password!";
    }
}
?>

```

p1.php — HTML Conditional Rendering

```

<!-- STATE 3: Render based on session -->
<?php if (isset($_SESSION["logged_in"]) && $_SESSION["logged_in"] === true): ?>
    <h2>Welcome, <?php echo htmlspecialchars($_SESSION["user"]); ?>!/h2>
    <p>You have accessed the secure area.</p>
    <a href="?action=logout">Logout</a>
<?php else: ?>
    <?php if ($error): ?>
        <div class="error"><?php echo $error; ?></div>
    <?php endif; ?>
    <form method="POST">
        <input type="text"      name="username" placeholder="Username" required>
        <input type="password" name="password" placeholder="Password" required>
        <button type="submit" name="login">Login</button>
    </form>
<?php endif; ?>

```

Problem 16: PHP Cookies vs Sessions Counter

Side-by-side comparison of session-based and cookie-based page visit counters.

Files	p1.php (single file — both counters + debug display)
Problem Statement	Display two counters that increment on every page load: one using PHP sessions (server-side, resets on browser close) and one using cookies (browser-side, persists across restarts for 30 days). Show the raw <code>\$_COOKIE</code> array for debugging.
Problem Approach	Session counter: increment <code>\$_SESSION["session_counter"]</code> on each load, initialising to 1 if not set. Cookie counter: read <code>\$_COOKIE["cookie_counter"]</code> , add 1 (default to 1 if not set), then call <code>setcookie()</code> to update the stored value with a 30-day expiry.

Solution Approach

session_start() and cookie logic both go BEFORE any HTML output. setcookie() sets the cookie for the NEXT request — the current request still sees the old value in \$_COOKIE, which is why \$cookie_val is computed separately and displayed. print_r(\$_COOKIE) dumps the raw cookie array for inspection.

Key PHP Concepts

session_start() | \$_SESSION["key"] | \$_COOKIE["key"] |
 setcookie(name, value, expiry, path) | time() — current Unix timestamp
 | time() + 86400*30 — 30 days from now | Ternary default: isset() ?
 val+1 : 1 | print_r() — debug dump

p1.php — Session & Cookie Counter Logic (before HTML)

```
<?php
session_start();    // must be first

// --- Session counter (server-side) ---
if (isset($_SESSION["session_counter"])) {
    $_SESSION["session_counter"]++;    // increment
} else {
    $_SESSION["session_counter"] = 1; // first visit this session
}

// --- Cookie counter (browser-side) ---
// Read existing value (+1), or start at 1 if cookie not set
$cookie_val = isset($_COOKIE["cookie_counter"])
    ? $_COOKIE["cookie_counter"] + 1
    : 1;

// Write updated value back to browser (30-day expiry)
// 86400 = seconds in a day, 86400*30 = 30 days
setcookie("cookie_counter", $cookie_val, time() + (86400 * 30), "/");
?>
```

p1.php — Display Both Counters + Debug

```
<h3>Session Counter (resets on browser close):</h3>
<p><?php echo $_SESSION["session_counter"]; ?></p>

<h3>Cookie Counter (persists 30 days):</h3>
<p><?php echo $cookie_val; ?></p>

<p><i>Refresh the page – both numbers increase!</i></p>

<h4>Raw $_COOKIE array (debug):</h4>
<pre><?php print_r($_COOKIE); ?></pre>
```

Problem 17: PHP File Upload with Validation

A file upload form with type and size validation, saving accepted files to an uploads/ directory.

Files	p1.php (single file — form + PHP upload handler) uploads/ (auto-created directory)
Problem Statement	Accept file uploads of type JPG, PNG, or PDF only, with a maximum size of 2MB. Show a success or error message. Create the uploads/ directory if it doesn't exist. Display the raw \$_FILES array for debugging.
Problem Approach	The form uses enctype='multipart/form-data' — required for file uploads. PHP reads the uploaded file from \$_FILES['myfile']. A validation chain checks: (1) upload error code, (2) MIME type against an allowed list, (3) file size. Only if all pass does move_uploaded_file() permanently save the file.
Solution Approach	Check \$file['error']!=0 first (catches PHP-level upload errors). in_array(\$file['type'], \$allowedTypes) validates MIME. \$file['size'] > \$maxSize rejects oversized files. is_dir(\$uploadDir) before mkdir() prevents errors if directory already exists. move_uploaded_file(\$file['tmp_name'], \$destination) moves from PHP's temp location to the permanent path.
Key PHP Concepts	\$_FILES['field'] — keys: name, type, size, tmp_name, error in_array(needle, haystack) is_dir(path) mkdir(path, permissions, recursive) move_uploaded_file(tmp, dest) basename(filename) — strip path enctype='multipart/form-data' on form

p1.php — HTML Form (enctype Required)

```
<!-- enctype="multipart/form-data" is MANDATORY for file uploads -->
<form action="" method="POST" enctype="multipart/form-data">
  <input type="file" name="myfile" required>
  <input type="submit" value="Upload Now">
</form>
```

p1.php — Full Validation & Upload Logic

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_FILES["myfile"])) {
    $file      = $_FILES["myfile"];
    $allowedTypes = ["image/jpeg", "image/png", "application/pdf"];
    $maxSize    = 2 * 1024 * 1024; // 2MB in bytes
    $uploadDir  = "uploads/";

    // Validation chain — earliest failure wins
    if ($file["error"] != 0) {
        $message = "Error: PHP upload failed (code " . $file["error"] . ")";
    } elseif (!in_array($file["type"], $allowedTypes)) {
        $message = "Error: Only JPG, PNG, PDF allowed.";
    } elseif ($file["size"] > $maxSize) {
        $message = "Error: File exceeds 2MB limit.";
    } else {
        // All checks passed — save the file
        if (!is_dir($uploadDir)) mkdir($uploadDir, 0777, true);
        $dest = $uploadDir . basename($file["name"]); // strip path from name
        if (move_uploaded_file($file["tmp_name"], $dest)) {
            $message = "Success! File saved to $dest";
        } else {
            $message = "Error: Could not save file.";
        }
    }
}
}
```

```
?>

<!-- Debug: show raw upload data -->
<?php if(isset($_FILES["myfile"])): ?>
    <pre><?php print_r($_FILES["myfile"]); ?></pre>
<?php endif; ?>
```

Problem 18: PHP MySQL Database Explorer (CRUD)

Two-page MySQL CRUD app — p1 explores databases/tables, p1a provides full Add/Edit/Delete on any table.

Files	p1.php (DB explorer: connect, list tables, preview data) p1a.php (CRUD: add, edit, delete rows) db.sql (schema for students table)
Problem Statement	p1: User enters a DB name; PHP lists its tables in a dropdown and previews the first 10 rows. p1a: Full CRUD — add new rows via form, edit existing rows (pre-filled form), delete rows via link. Column names are read dynamically from SHOW COLUMNS.
Problem Approach	p1: new mysqli() connects. select_db() switches DB. SHOW TABLES lists tables. SELECT * FROM table LIMIT 10 previews data. Column headers are read dynamically from the first fetch_assoc() row. p1a: Three POST/GET operations share one page. SHOW COLUMNS reads column names to build a dynamic form. edit mode pre-fills form from a SELECT WHERE id.
Solution Approach	p1: POST with db_name triggers table listing. Selecting a table name auto-submits (onchange). fetch_assoc() returns associative arrays keyed by column name. p1a: DELETE via GET ?delete=id. INSERT via POST 'add' using implode(',', \$cols) and implode(',', \$vals). UPDATE via POST 'update' building a SET clause with implode. Edit mode queries the row by id.
Key PHP & SQL	new mysqli(host,user,pass,db,port) \$conn->select_db() \$conn->query(sql) \$res->fetch_array() \$res->fetch_assoc() implode(glue, array) SHOW TABLES SHOW COLUMNS FROM table SELECT * FROM t LIMIT 10 INSERT INTO t (cols) VALUES (vals) UPDATE t SET col=val WHERE id=n DELETE FROM t WHERE id=n htmlspecialchars() header('Location:')

db.sql — Table Schema

```
CREATE TABLE students (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    name    VARCHAR(100) NOT NULL,
    email   VARCHAR(100) NOT NULL
);
```

p1.php — Connect, List Tables, Preview Data

```
<?php
$host="localhost"; $user="root"; $pass=""; $port=3307;
$conn = new mysqli($host, $user, $pass, "", $port);
```

```

$db_name = $_POST["db_name"] ?? "";
$selected_table = $_POST["table_name"] ?? "";
$tables = [];

if ($db_name) {
    $conn->select_db($db_name);           // switch to user DB
    $res = $conn->query("SHOW TABLES");
    while ($row = $res->fetch_array())
        $tables[] = $row[0];           // collect table names
}
?>
<!-- Dropdown auto-submits on selection -->
<select name="table_name" onchange="this.form.submit()">
    <?php foreach ($tables as $t): ?>
        <option value="<?=$t?>" <?=$selected_table==$t?"selected":"" ?>><?=$t?></option>
    <?php endforeach; ?>
</select>

<?php if ($selected_table): ?>
    <?php
        $conn->select_db($db_name);
        $data = $conn->query("SELECT * FROM $selected_table LIMIT 10");
        $first = true;
        while ($row = $data->fetch_assoc()):
            if ($first) { // first row: generate <th> from keys
                foreach ($row as $col => $v) echo "<th>$col</th>";
                $first = false;
            }
            foreach ($row as $v) echo "<td>$v</td>";
        endwhile;
    ?>
<?php endif; ?>

```

p1a.php — Dynamic CRUD (Insert / Update / Delete)

```

// DELETE: triggered by ?delete=id in URL
if (isset($_GET["delete"])) {
    $id = $_GET["delete"];
    $conn->query("DELETE FROM $table WHERE id=$id");
}

// INSERT: triggered by POST "add" button
if (isset($_POST["add"])) {
    $cols = []; $vals = [];
    foreach ($_POST["data"] as $col => $val) {
        $cols[] = $col;
        $vals[] = "'$val'";
    }
    $conn->query("INSERT INTO $table (" . implode(",", $cols) . ")
        VALUES (" . implode(",", $vals) . ")");
}

// UPDATE: triggered by POST "update" button
if (isset($_POST["update"])) {
    $id = $_POST["id"];
    $updates = [];
    foreach ($_POST["data"] as $col => $val)
        $updates[] = "$col='$val'";
    $conn->query("UPDATE $table SET " . implode(",", $updates) . " WHERE id=$id");
}

```

```

}

// EDIT MODE: pre-fill form by reading the row
if (isset($_GET["edit"])) {
    $id = $_GET["edit"];
    $edit_row = $conn->query("SELECT * FROM $table WHERE id=$id")->fetch_assoc();
}

// Dynamic form: columns from SHOW COLUMNS
$res = $conn->query("SHOW COLUMNS FROM $table");
while ($row = $res->fetch_assoc())
    if ($row["Field"] != "id") $columns[] = $row["Field"];

```

Problem 19: PHP File-Based Attendance System

A three-page attendance tracker that stores records in a plain text file — no database required.

Files	p1.php (main menu) p1a.php (add attendance record) p1b.php (attendance report) a.txt (data store)
Problem Statement	Build a system where teachers can: (1) add student attendance records (date, roll, name, present/absent) which are saved to a file, and (2) view an attendance report showing each student's present and absent counts.
Problem Approach	p1a.php: Reads form data and appends a formatted line to a.txt using <code>file_put_contents</code> with <code>FILE_APPEND</code> . p1b.php: Reads a.txt line by line using <code>file()</code> , splits each line with <code>explode()</code> , and builds an associative array of student stats. Renders the stats table in a loop.
Solution Approach	p1a: Line format is 'date roll name status\n'. <code>file_put_contents('a.txt', \$line, FILE_APPEND)</code> adds without overwriting. p1b: <code>file('a.txt', FILE_IGNORE_NEW_LINES FILE_SKIP_EMPTY_LINES)</code> returns clean array of lines. <code>explode(' ', \$line)</code> splits into 4 parts. <code>\$studentStats[\$name]['p']++</code> increments present count per student. <code>file_exists()</code> check prevents crash if a.txt is missing.
Key PHP Concepts	<code>file_put_contents(file, data, FLAG)</code> <code>FILE_APPEND</code> flag <code>file(path, flags)</code> — reads into array <code>FILE_IGNORE_NEW_LINES</code> <code>FILE_SKIP_EMPTY_LINES</code> <code>explode(delimiter, string)</code> — split <code>trim()</code> — remove whitespace <code>strtolower()</code> — case-insensitive compare <code>file_exists()</code> <code>count()</code> — array length Associative arrays for stat tracking <code>date('Y-m-d')</code> — today's date

p1a.php — Appending Attendance Records to File

```

<?php
$success = false;
if (isset($_POST["submit"])) {
    $date   = $_POST["date"];
    $roll   = $_POST["roll"];
    $name    = $_POST["name"];
    $status = $_POST["status"];

```

```
// Build one line in pipe-delimited format
$line = "$date | $roll | $name | $status\n";

// FILE_APPEND adds to end without overwriting
file_put_contents("a.txt", $line, FILE_APPEND);
$success = true;
}
?>
<!-- Default date = today -->


```

p1b.php — Reading & Aggregating Attendance Stats

```
<?php
$studentStats = [];

if (file_exists("a.txt")) {
    // Read file into array, skip blank lines, strip newlines
    $lines = file("a.txt", FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);

    foreach ($lines as $line) {
        // Split "2026-04-22 | 101 | Nitya | Present"
        $parts = explode(" | ", $line);
        if (count($parts) == 4) {
            $name = trim($parts[2]);
            $status = trim($parts[3]);

            // Initialise student entry if first time seen
            if (!isset($studentStats[$name]))
                $studentStats[$name] = ["p" => 0, "a" => 0];

            if (strtolower($status) == "present")
                $studentStats[$name]["p"]++;
            else
                $studentStats[$name]["a"]++;
        }
    }
}
?>
<!-- Render report table -->
<p>Total students: <?= count($studentStats) ?></p>
<table>
    <tr><th>Name</th><th>Present</th><th>Absent</th></tr>
    <?php foreach ($studentStats as $name => $data): ?>
        <tr>
            <td><?= htmlspecialchars($name) ?></td>
            <td><?= $data["p"] ?></td>
            <td><?= $data["a"] ?></td>
        </tr>
    <?php endforeach; ?>
</table>
```

Part 2: HTML — Complete Study Notes

HTML (HyperText Markup Language) is the backbone of every webpage. It defines structure and content using tags. It is NOT a programming language — it has no logic or variables. The browser reads HTML top-to-bottom and renders it visually.

Chapter 1.1 — Document Structure

Every valid HTML document follows this skeleton:

```
<!DOCTYPE html>          <!-- Required: tells browser this is HTML5 -->
<html lang="en">          <!-- Root element; lang helps screen readers -->
  <head>
    <meta charset="UTF-8">      <!-- character encoding -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Page description for search engines">
    <title>My Page Title</title> <!-- appears in browser tab -->
    <link rel="stylesheet" href="style.css">
    <link rel="icon" href="favicon.ico">
  </head>
  <body>
    <!-- All visible content goes here -->
  </body>
</html>
```

NOTE: Everything inside `<head>` is metadata — it configures the page but is not visible. Everything inside `<body>` is rendered on screen.

Chapter 1.2 — Text Tags

Headings, paragraphs and inline text formatting are the most used HTML elements.

```
<h1>Most Important Heading</h1>    <!-- only one per page -->
<h2>Section Title</h2>
<h3>Sub-section</h3>                <!-- h1 through h6 -->
<p>This is a paragraph.</p>
<b>Bold text</b> or <strong>Semantic bold</strong>
<i>Italic text</i> or <em>Semantic italic (emphasis)</em>
<u>Underlined</u>
<s>Strikethrough</s>
<sup>Superscript</sup> e.g. x<sup>2</sup> → x2
<sub>Subscript</sub> e.g. H<sub>2</sub>O → H2O
<small>Smaller text</small>
<mark>Highlighted text</mark>
<br>    <!-- line break — self-closing -->
<hr>    <!-- horizontal rule / divider line — self-closing -->
```

TIP: Use `<strong>` and `<em>` instead of `<b>` and `<i>` for better accessibility — they carry semantic meaning (importance/emphasis), not just visual styling.

Chapter 1.3 — HTML5 Semantic Elements

HTML5 introduced semantic tags that describe the meaning of content, improving SEO and accessibility. Use these instead of generic `<div>` wherever appropriate.

```
<header>    - top of page or section (logo, nav, title)</header>
<nav>       - navigation links</nav>
<main>      - primary content area (one per page)</main>
<section>   - themed grouping of content</section>
<article>   - self-contained content (blog post, card)</article>
<aside>     - sidebar or related content</aside>
<footer>    - bottom of page (copyright, links)</footer>
<figure>    - image or chart with caption
  
  <figcaption>Q1 2026 Sales</figcaption>
</figure>
<details>   - collapsible section
  <summary>Click to expand</summary>
  Hidden content here...
</details>
```

Chapter 1.4 — Links & Navigation

```
<!-- External link, new tab -->
<a href="https://google.com" target="_blank" rel="noopener">Google</a>

<!-- Internal page link -->
<a href="about.html">About Us</a>

<!-- Same-page anchor navigation -->
<a href="#contact">Jump to Contact</a>
<h2 id="contact">Contact Section</h2>    <!-- destination -->

<!-- Email link -->
<a href="mailto:name@example.com">Email Me</a>

<!-- Phone link (mobile) -->
<a href="tel:+919876543210">Call Us</a>

<!-- Image as a link -->
<a href="https://example.com">
  
</a>
```

Chapter 1.5 — Images

```
<!-- Basic image -->


<!-- Responsive image (scales with screen) -->


<!-- Image with border and alignment -->


<!-- Lazy loading (loads only when scrolled into view) -->

```

NOTE: The alt attribute is critical — it is read by screen readers and shown when the image fails to load. Always write meaningful alt text.

Chapter 1.6 — Lists (All Three Types + Nesting)

```
<!-- Unordered list (bullets) -->
<ul type="disc">    <!-- type: disc (default), circle, square -->
  <li>HTML</li>
  <li>CSS</li>
</ul>

<!-- Ordered list (numbers) -->
<ol type="1" start="3">  <!-- type: 1,A,a,I,i | start from 3 -->
  <li>First item (shows as 3)</li>
  <li>Second item (shows as 4)</li>
</ol>

<!-- Definition list -->
<dl>
  <dt>HTML</dt>    <!-- term -->
  <dd>HyperText Markup Language</dd>    <!-- definition -->
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
</dl>

<!-- Nested list -->
<ul>
  <li>Programming Languages
    <ul>      <!-- nested inside parent li -->
      <li>Python</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Databases</li>
</ul>
```

Chapter 1.7 — Tables (Full Reference)

```
<table border="1" cellpadding="8" cellspacing="0" width="100%">
  <caption>Student Marks Table</caption>    <!-- optional title -->
  <thead>
    <tr>
      <th>Name</th>
      <th colspan="2">Scores</th>    <!-- merges 2 columns -->
    </tr>
  </thead>
  <tbody>
    <tr>
      <td rowspan="2">Alice</td>    <!-- merges 2 rows -->
      <td>Maths: 90</td>
      <td>Science: 85</td>
    </tr>
    <tr>
      <!-- Alice cell merged from above -->
      <td>English: 78</td>
      <td>History: 82</td>
```

```

    </tr>
  </tbody>
</tfoot>
  <tr><td colspan="3">End of Table</td></tr>
</tfoot>
</table>

```

TIP: When using rowspan/colspan, reduce the number of <td>/<th> in affected rows accordingly. Total column count must stay consistent in every row.

Chapter 1.8 — Forms (Complete Reference)

Forms collect user data and send it to a server or handle it with JavaScript.

```

<form action="handler.php" method="post" enctype="multipart/form-data">

  <!-- Text inputs -->
  <label for="name">Full Name:</label>
  <input type="text" id="name" name="name" placeholder="John Doe" required>

  <input type="email"      name="email"      placeholder="you@example.com" required>
  <input type="password"   name="password"   minlength="8">
  <input type="number"     name="age"         min="18" max="100" step="1">
  <input type="tel"        name="phone"       pattern="[0-9]{10}">
  <input type="url"         name="website"    placeholder="https://...">
  <input type="date"       name="dob">
  <input type="time"       name="meeting">
  <input type="color"      name="fav_color">
  <input type="range"      name="rating" min="1" max="10" value="5">
  <input type="search"     name="query"       placeholder="Search...">
  <input type="file"       name="resume"      accept=".pdf,.doc">
  <input type="hidden"     name="token"       value="abc123">

  <!-- Radio buttons (same name = one group) -->
  <input type="radio" name="gender" value="male"  > Male
  <input type="radio" name="gender" value="female"> Female

  <!-- Checkboxes -->
  <input type="checkbox" name="hobbies" value="reading"> Reading
  <input type="checkbox" name="hobbies" value="coding"> Coding

  <!-- Dropdown -->
  <select name="course" required>
    <option value="" disabled selected>-- Select --</option>
    <option value="btech">B.Tech</option>
    <option value="mtech" selected>M.Tech</option>  <!-- pre-selected -->
  </select>

  <!-- Multi-select -->
  <select name="skills" multiple size="4">
    <option value="html">HTML</option>
    <option value="css">CSS</option>
  </select>

  <!-- Textarea -->
  <textarea name="bio" rows="5" cols="40"
    minlength="20" maxlength="500"

```

```

placeholder="Write something..."></textarea>

<!-- Buttons -->
<input type="submit" value="Submit">
<input type="reset" value="Clear">
<button type="button" onclick="doSomething()">Custom Action</button>

</form>

```

Chapter 1.9 — HTML5 Validation Attributes

required	Field must be filled before submit
pattern='...'	Value must match the regex. e.g. pattern='[A-Z]{3}[0-9]{4}' (3 uppercase letters + 4 digits)
minlength / maxlength	Character count limits for text inputs and textareas
min / max	Numeric/date range limits for number and date inputs
step	Increment for number inputs. step='0.5' allows halves
title	Tooltip shown when validation fails — explains the required format
placeholder	Ghost text shown when field is empty — disappears on typing
autocomplete	autocomplete='off' disables browser autofill on sensitive fields

Chapter 1.10 — HTML Entities & Special Characters

Some characters have special meaning in HTML (< > & " ') and must be escaped using entities.

&lt;	< (less than) — use in text to show < without opening a tag
&gt;	> (greater than)
&amp;	& (ampersand) — must be escaped in URLs and text
&quot;	" (double quote)
&apos;	' (apostrophe)
&nbsp;	Non-breaking space — prevents line break between words
&copy;	© copyright symbol
&reg;	® registered trademark
&trade;	™ trademark
&#10004;	✓ checkmark (used in p6 confirmation page)
&#9733;	★ star
&hearts;	♥ heart

Chapter 1.11 — Multimedia: Audio, Video, iFrame

```
<!-- Video with controls -->
<video width="640" height="360" controls autoplay muted loop>
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser does not support the video tag.
</video>

<!-- Audio player -->
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
</audio>

<!-- Embed external page (e.g. Google Maps) -->
<iframe src="https://maps.google.com/..." width="600" height="400"
  frameborder="0" allowfullscreen></iframe>
```

Part 2: CSS — Complete Study Notes

CSS (Cascading Style Sheets) controls the visual presentation of HTML. Without CSS a page is plain text on a white background. CSS defines colours, fonts, spacing, layout, animations, and responsive behaviour.

Chapter 2.1 — Three Ways to Apply CSS

```
<!-- 1. Inline — directly on element (highest specificity, avoid for maintenance) -->
<p style="color:red; font-size:20px;">Red paragraph</p>

<!-- 2. Internal — inside <style> block in <head> (page-specific) -->
<head>
  <style>
    p { color: blue; font-size: 18px; }
  </style>
</head>

<!-- 3. External — separate .css file (best: reusable across pages) -->
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

Chapter 2.2 — CSS Specificity & the Cascade

When multiple rules target the same element, the browser uses specificity to decide which wins. Higher specificity overrides lower specificity. Specificity is calculated as (inline, ID, class, element).

Inline style (style=")	Specificity: 1,0,0,0 — highest, always wins over stylesheets
ID selector (#id)	Specificity: 0,1,0,0
Class / attribute / pseudo-class	Specificity: 0,0,1,0
Element (tag) selector	Specificity: 0,0,0,1 — lowest
!important	Overrides everything — use sparingly, breaks the cascade
Cascade rule (same specificity)	The LAST rule defined wins

```
/* specificity: 0,0,0,1 */
p { color: black; }
/* specificity: 0,0,1,0 — wins over above */
.intro { color: blue; }
/* specificity: 0,1,0,0 — wins over class */
#main { color: green; }
/* !important — wins over everything */
p { color: red !important; }
```

Chapter 2.3 — Selectors (All Types)

```

/* Element */
p { } h1 { } input { }

/* Class (.) – can apply to multiple elements */
.card { } .btn-primary { }

/* ID (#) – unique per page */
#header { } #login-form { }

/* Group – apply same rule to multiple selectors */
h1, h2, h3 { font-family: Arial; }

/* Descendant – any nested level */
div p { } /* any <p> inside any <div> */

/* Child combinator (>) – direct children only */
ul > li { } /* only immediate <li> children of <ul> */

/* Adjacent sibling (+) – immediately after */
h2 + p { } /* first <p> immediately after an <h2> */

/* General sibling (~) – all siblings after */
h2 ~ p { } /* all <p> elements after any <h2> */

/* Attribute selectors */
input[type="text"] { } /* input whose type is text */
a[href^="https"] { } /* href starts with https */
a[href$=".pdf"] { } /* href ends with .pdf */
a[href*="google"] { } /* href contains "google" */

/* Pseudo-classes – element state */
a:hover { } /* mouse is over it */
a:visited { } /* already clicked */
a:active { } /* being clicked */
input:focus { } /* keyboard focused */
input:disabled { }
li:first-child { }
li:last-child { }
li:nth-child(2) { } /* 2nd child */
li:nth-child(odd) { } /* 1st,3rd,5th... */
li:nth-child(even) { } /* 2nd,4th,6th... */
li:nth-child(3n+1) { } /* every 3rd starting at 1 */
:not(.active) { } /* elements NOT matching selector */

/* Pseudo-elements – style part of element */
p::first-letter { font-size: 200%; } /* first letter only */
p::first-line { font-weight: bold; }
div::before { content: ">> "; color: red; } /* insert before */
div::after { content: "<<"; color: red; } /* insert after */

```

Chapter 2.4 — The Box Model

Every HTML element is a rectangular box. Understanding the box model is the foundation of all CSS layout.

```

.box {
  width: 300px; /* content width */

```

```

height: 150px;          /* content height */
padding: 20px;          /* space INSIDE border (all 4 sides) */
border: 3px solid red;  /* the visible edge */
margin: 15px;           /* space OUTSIDE border (all 4 sides) */
}
/* Total rendered width = width + padding*2 + border*2 + margin*2 */
/*                               = 300 + 40 + 6 + 30 = 376px */

/* Shorthand for all 4 sides: top right bottom left (clockwise) */
padding: 10px 20px 10px 20px;
margin: 5px 10px;       /* top/bottom=5px, left/right=10px */

/* box-sizing: border-box — width includes padding and border */
* { box-sizing: border-box; } /* most modern reset */
/* Now: width:300px means total box is 300px including padding+border */

```

Chapter 2.5 — Typography

```

body {
  font-family: "Segoe UI", Arial, sans-serif; /* fallback chain */
  font-size: 16px; /* base size */
  font-weight: normal; /* normal(400), bold(700), 100-900 */
  font-style: normal; /* italic | oblique */
  line-height: 1.6; /* 1.6x the font-size — good readability */
  color: #333333;
  text-align: left; /* left | center | right | justify */
}
h1 {
  font-size: 2.5rem; /* rem = relative to root font-size */
  letter-spacing: 2px; /* space between characters */
  word-spacing: 5px;
  text-transform: uppercase; /* uppercase | lowercase | capitalize */
  text-decoration: underline; /* underline | overline | line-through | none */
  text-shadow: 2px 2px 4px rgba(0,0,0,0.3); /* h-offset v-offset blur color */
}
/* Google Font import (add to top of CSS file) */
@import url("https://fonts.googleapis.com/css2?family=Poppins&display=swap");
body { font-family: "Poppins", sans-serif; }

```

Chapter 2.6 — Colours & Backgrounds

```

/* Colour formats */
color: red; /* named colour */
color: #FF5733; /* hex: #RRGGBB */
color: #F53; /* short hex: same as #FF5533 */
color: rgb(255, 87, 51); /* red, green, blue 0-255 */
color: rgba(255, 87, 51, 0.5); /* + alpha 0-1 (transparent-opaque) */
color: hsl(14, 100%, 60%); /* hue(0-360), saturation%, lightness% */
color: hsla(14, 100%, 60%, 0.7);

/* Background */
background-color: #1A237E;
background-image: url("bg.jpg");
background-size: cover; /* cover | contain | 100% 200px */
background-position: center; /* center | top | 50% 50% */
background-repeat: no-repeat; /* no-repeat | repeat | repeat-x */
background-attachment: fixed; /* parallax-like fixed background */

```

```

/* Shorthand */
background: url("bg.jpg") no-repeat center/cover;

/* Gradient backgrounds */
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
background: radial-gradient(circle, #ff9a9e, #fad0c4);

```

Chapter 2.7 — Display & Visibility

display: block	Full width, new line. Default for div, p, h1-h6, section, header
display: inline	Flows with text, no width/height. Default for span, a, strong, em
display: inline-block	Inline flow + can set width/height. Useful for buttons
display: none	Completely removes from layout — no space taken
display: flex	Flexbox container — children become flex items
display: grid	CSS Grid container — powerful 2D layout
display: table	Behaves like HTML table
visibility: hidden	Invisible but still takes space in layout
opacity: 0	Fully transparent but still takes space and is clickable

Chapter 2.8 — Flexbox Layout

Flexbox is the go-to tool for one-dimensional layout (row or column). Set `display:flex` on the parent — all direct children become flex items.

```

/* Parent (flex container) properties */
.container {
  display:          flex;
  flex-direction:   row;           /* row (default) | column | row-reverse | column-
reverse */
  justify-content:   center;       /* main axis: flex-start | flex-end | center | space-
between | space-around | space-evenly */
  align-items:       center;       /* cross axis: stretch | flex-start | flex-end | center
| baseline */
  flex-wrap:         wrap;         /* nowrap (default) | wrap | wrap-reverse */
  gap:               20px;        /* space between items (replaces margin hacks) */
}

/* Child (flex item) properties */
.item {
  flex-grow: 1;    /* how much extra space to take (1=equal share) */
  flex-shrink: 1;  /* how much to shrink when space is tight */
  flex-basis: 200px; /* starting size before grow/shrink */
  flex: 1 1 200px; /* shorthand: grow shrink basis */
  align-self: flex-start; /* override align-items for this item */
  order: 2;             /* change visual order without changing HTML */
}

/* Common patterns */
.nav { display:flex; justify-content:space-between; align-items:center; }

```

```
.cards { display:flex; flex-wrap:wrap; gap:20px; }
.center { display:flex; justify-content:center; align-items:center; height:100vh; }
```

Chapter 2.9 — CSS Grid Layout

Grid is the most powerful CSS layout system, designed for two-dimensional (rows AND columns) layouts.

```
/* Define a grid */
.grid-container {
  display: grid;
  grid-template-columns: 200px 1fr 1fr; /* 3 cols: fixed + 2 equal flex */
  grid-template-rows: 100px auto 60px; /* 3 rows */
  gap: 15px; /* space between cells */
}

/* Repeat shorthand */
grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); /* responsive! */

/* Place items in specific cells */
.header {
  grid-column: 1 / 4; /* spans from col-line 1 to 4 (all 3 columns) */
  grid-row: 1 / 2;
}
.sidebar { grid-column: 1 / 2; grid-row: 2 / 3; }
.content { grid-column: 2 / 4; grid-row: 2 / 3; }
.footer { grid-column: 1 / 4; grid-row: 3 / 4; }
```

Chapter 2.10 — Positioning

position: static (default)	Normal flow. top/left/right/bottom have NO effect
position: relative	Normal flow position, THEN offset with top/left/right/bottom. Other elements are NOT affected
position: absolute	Removed from flow. Positioned relative to nearest non-static ancestor (or <body>)
position: fixed	Removed from flow. Stays fixed in viewport — doesn't scroll. Good for navbars, modals
position: sticky	Normal flow until scrolled to threshold, then sticks. Needs top/left defined

```
.parent { position: relative; } /* creates positioning context */
.child {
  position: absolute;
  top: 10px; /* 10px from parent top */
  right: 20px; /* 20px from parent right */
}
.navbar { position: fixed; top:0; left:0; width:100%; z-index:1000; }
.sticky-header { position: sticky; top: 0; }
```

Chapter 2.11 — CSS Variables (Custom Properties)

```
:root {                                /* define on :root = global scope */
  --primary-color: #1A237E;
  --font-size-base: 16px;
  --border-radius: 8px;
  --spacing-md: 20px;
}

.button {
  background-color: var(--primary-color);
  font-size:      var(--font-size-base);
  border-radius:   var(--border-radius);
  padding:        var(--spacing-md);
}

/* Override in a local scope */
.dark-theme {
  --primary-color: #90CAF9; /* lighter shade for dark backgrounds */
}
```

Chapter 2.12 — Responsive Design & Media Queries

```
/* Mobile-first approach: default styles for small screens */
.card { width: 100%; }

/* Tablet: 600px and wider */
@media (min-width: 600px) {
  .card { width: 48%; }
}

/* Desktop: 1024px and wider */
@media (min-width: 1024px) {
  .card { width: 31%; }
}

/* Common breakpoints */
@media (max-width: 480px) { /* phone */ }
@media (max-width: 768px) { /* tablet */ }
@media (max-width: 1200px) { /* laptop */ }

/* Orientation */
@media (orientation: landscape) { }

/* Print styles */
@media print { .no-print { display: none; } }
```

Chapter 2.13 — CSS Transitions & Animations

```
/* Transition: smooth change when a property changes */
.button {
  background-color: green;
  transition: background-color 0.3s ease,
              transform      0.2s ease-in-out;
}

.button:hover {
  background-color: darkgreen;
  transform: scale(1.05); /* 5% bigger on hover */
}
```

```
}

/* Keyframe animation */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(-20px); }
  to   { opacity: 1; transform: translateY(0); }
}

.hero {
  animation: fadeIn 0.8s ease forwards;
  /* name duration easing fill-mode */
}

@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

.loader { animation: spin 1s linear infinite; }
```

Chapter 2.14 — Shadows, Borders & Border-Radius

```
/* Box shadow */
box-shadow: h-offset v-offset blur spread color;
.card { box-shadow: 0 4px 12px rgba(0,0,0,0.15); }
.pressed { box-shadow: inset 0 2px 4px rgba(0,0,0,0.3); } /* inset = inner */

/* Text shadow */
h1 { text-shadow: 2px 2px 5px rgba(0,0,0,0.4); }

/* Border shorthand */
border: 2px solid #333;
border-top: 3px dashed red;
border-bottom: 1px dotted grey;
border: none;

/* Border-radius */
border-radius: 8px; /* all 4 corners */
border-radius: 50%; /* perfect circle (on square element) */
border-radius: 20px 0 20px 0; /* top-left bottom-right | top-right bottom-left */
```

Part 2: JavaScript — Complete Study Notes

JavaScript (JS) is the only programming language that runs natively in the browser. It makes pages interactive — reacting to clicks, validating forms, fetching data, updating content without reloading, and much more.

Chapter 3.1 — Variables & Scope

```
// var - function-scoped, hoisted, avoid in modern code
var x = 10;

// let - block-scoped, reassignable
let count = 0;
count = 5;           // OK

// const - block-scoped, cannot be reassigned
const PI = 3.14159;
// PI = 3;           // ERROR - cannot reassign
const arr = [1,2,3];
arr.push(4);         // OK - we changed contents, not the reference

// Block scope demonstration
{
  let blockVar = "inside";
  var funcVar  = "anywhere";
}
// console.log(blockVar); // ERROR: not accessible
console.log(funcVar);    // OK: var leaks out of blocks
```

Chapter 3.2 — Data Types & Type Coercion

```
// Primitive types
typeof "hello"    // "string"
typeof 42         // "number"
typeof 3.14       // "number" (no separate float type)
typeof true       // "boolean"
typeof undefined  // "undefined"
typeof null       // "object" (historical bug in JS)
typeof Symbol()   // "symbol"

// Type coercion (automatic conversion)
"5" + 3          // "53" - number coerced to string (+concatenates)
"5" - 3          // 2   - string coerced to number (-subtracts)
"5" * "2"        // 10  - both coerced to numbers

// Strict equality avoids coercion
"5" == 5         // true  (loose: coerces types)
"5" === 5        // false (strict: different types)
0 == false       // true  (loose)
0 === false      // false (strict) - always use ===

// Falsy values (evaluate as false in conditions)
```

```
// false, 0, "", null, undefined, NaN
if (!value) { /* value is falsy */ }
```

Chapter 3.3 — Functions (All Styles)

```
// Function declaration (hoisted – can call before definition)
function greet(name) {
  return "Hello, " + name;
}
console.log(greet("Nitya")); // "Hello, Nitya"

// Function expression (not hoisted)
const add = function(a, b) {
  return a + b;
};

// Arrow function (ES6 – shorter syntax)
const multiply = (a, b) => a * b; // implicit return
const square = n => n * n; // one param, no brackets needed
const hello = () => console.log("Hi!"); // no params

// Default parameters
function power(base, exp=2) { // exp defaults to 2
  return base ** exp;
}
power(3); // 9 (3²)
power(3, 3); // 27 (3³)

// Rest parameters (variable arguments)
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4); // 10
```

Chapter 3.4 — Arrays & Key Methods

```
const fruits = ["apple", "banana", "cherry"];

// Access & modify
fruits[0] // "apple"
fruits.length // 3
fruits.push("date") // adds to end, returns new length
fruits.pop() // removes and returns last
fruits.unshift("avocado") // adds to beginning
fruits.shift() // removes and returns first
fruits.splice(1, 1) // remove 1 element at index 1
fruits.splice(1, 0, "blueberry") // insert at index 1

// Search
fruits.indexOf("banana") // 1 (or -1 if not found)
fruits.includes("cherry") // true
fruits.find(f => f.startsWith("c")) // "cherry"
fruits.findIndex(f => f === "banana") // 1

// Transform (return NEW array – original unchanged)
fruits.map(f => f.toUpperCase()) // ["APPLE", "BANANA", "CHERRY"]
```

```
fruits.filter(f => f.length > 5)           // ["banana","cherry"]
fruits.slice(1, 3)                         // ["banana","cherry"]
[1,2,3].reduce((acc, n) => acc + n, 0) // 6 (sum)

// Sort & reverse
[3,1,4,1,5].sort((a,b) => a - b) // [1,1,3,4,5] ascending
[3,1,4,1,5].sort((a,b) => b - a) // [5,4,3,1,1] descending
fruits.reverse()

// Spread and joining
const all = [...fruits, "elderberry"]; // spread into new array
fruits.join(", ") // "apple, banana, cherry"
```

Chapter 3.5 — Objects

```
const student = {
  name: "Nitya",
  age: 21,
  cgpa: 9.5,
  greet: function() { return "Hi, I am " + this.name; }
};

// Access
student.name // "Nitya"
student["age"] // 21 (bracket notation - works with dynamic keys)
student.greet() // "Hi, I am Nitya"

// Add / update / delete properties
student.email = "n@example.com";
student.name = "Nish";
delete student.cgpa;

// Destructuring (ES6)
const { name, age } = student;
console.log(name, age); // "Nish" 21

// Spread merge
const updated = { ...student, age: 22 }; // shallow copy with override

// Object methods
Object.keys(student) // ["name","age","email","greet"]
Object.values(student) // ["Nish",21,"n@example.com",fn]
Object.entries(student) // [["name","Nish"],["age",21],...]
```

Chapter 3.6 — DOM Manipulation (Complete)

```
// — Selecting elements —
document.getElementById("myId") // single by ID
document.querySelector(".myClass") // first match (CSS selector)
document.querySelectorAll("p.intro") // NodeList of all matches
document.getElementsByTagName("div") // HTMLCollection by tag
document.getElementsByClassName("card") // HTMLCollection by class

// — Reading & writing content —
el.textContent = "Plain text"; // set plain text (safe, no HTML)
el.innerHTML = "<b>Bold</b>"; // set HTML (XSS risk if user-supplied)
```

```

el.innerText = "Visible text"; // like textContent but respects CSS visibility

// — Form input values —
document.getElementById("myInput").value; // read
document.getElementById("myInput").value = "new"; // write

// — Attributes —
el.getAttribute("src")
el.setAttribute("src", "new-image.jpg")
el.removeAttribute("disabled")
el.hasAttribute("required") // true/false

// — CSS styles —
el.style.backgroundColor = "red";
el.style.fontSize = "20px";
el.style.display = "none";

// — CSS classes —
el.classList.add("active");
el.classList.remove("active");
el.classList.toggle("active"); // add if missing, remove if present
el.classList.contains("active"); // true/false

// — Creating & inserting elements —
const div = document.createElement("div");
div.textContent = "New content";
div.className = "card";
document.body.appendChild(div); // add at end
parent.insertBefore(div, sibling); // insert before sibling
parent.removeChild(child); // remove
el.remove(); // modern: remove self

```

Chapter 3.7 — Events (Complete)

```

// addEventListener (preferred – separates HTML from JS)
el.addEventListener("click", function(event) {
  console.log("Clicked!", event.target);
});

// Arrow function listener
btn.addEventListener("click", (e) => {
  e.preventDefault(); // prevent default action (e.g. form submit, link follow)
  e.stopPropagation(); // stop event bubbling up to parent elements
});

// Remove listener
function handler() { console.log("hi"); }
el.addEventListener("click", handler);
el.removeEventListener("click", handler); // must use same function reference

// Common events:
// Mouse: click, dblclick, mousedown, mouseup, mouseover, mouseout, mousemove
// Keyboard: keydown, keyup, keypress
// Form: submit, change, input, focus, blur, reset
// Window: load, DOMContentLoaded, resize, scroll
// Touch: touchstart, touchend, touchmove

```

```
// Event delegation – one listener handles many children
document.getElementById("list").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    e.target.classList.toggle("done");
  }
});
```

Chapter 3.8 — Error Handling

```
try {
  let result = riskyFunction();
  JSON.parse("invalid json"); // throws SyntaxError
  eval("1+"); // throws SyntaxError
} catch (error) {
  console.error("Error:", error.message);
  console.error("Type:", error.name); // SyntaxError, TypeError, etc.
} finally {
  console.log("This runs regardless");
}

// Throw custom errors
function divide(a, b) {
  if (b === 0) throw new Error("Cannot divide by zero");
  return a / b;
}
```

Chapter 3.9 — Timers & Async

```
// setTimeout – runs ONCE after delay
const id1 = setTimeout(() => {
  console.log("Runs after 2 seconds");
}, 2000);
clearTimeout(id1); // cancel before it fires

// setInterval – runs REPEATEDLY
const id2 = setInterval(() => {
  console.log("Every second");
}, 1000);
clearInterval(id2); // stop

// Promises (async operations)
const promise = new Promise((resolve, reject) => {
  // async work...
  if (success) resolve("Done!");
  else reject(new Error("Failed"));
});
promise.then(result => console.log(result))
  .catch(err => console.error(err));

// async/await (cleaner promise syntax)
async function fetchData() {
  try {
    const response = await fetch("https://api.example.com/data");
    const data = await response.json();
    console.log(data);
  } catch (err) {
```

```
    console.error("Fetch failed:", err);
  }
}
```

Chapter 3.10 — JSON & Local Storage

```
// JSON (JavaScript Object Notation) – data interchange format
const obj = { name: "Nitya", score: 95 };
const json = JSON.stringify(obj);    // convert to string: {"name":"Nitya","score":95}
const back = JSON.parse(json);       // convert back to object

// localStorage – persists across sessions (survives browser close)
localStorage.setItem("theme", "dark");
localStorage.getItem("theme");        // "dark"
localStorage.removeItem("theme");
localStorage.clear();

// Store objects in localStorage (must serialise to JSON)
localStorage.setItem("user", JSON.stringify({ name:"Nitya" }));
const user = JSON.parse(localStorage.getItem("user"));

// sessionStorage – same API, but cleared when tab closes
sessionStorage.setItem("token", "abc123");
```

Chapter 3.11 — ES6+ Modern Features

```
// Template literals
const name = "Nitya";
const msg = `Hello, ${name}! Score: ${90 + 5}.`; // backtick string

// Destructuring
const [a, b, ...rest] = [1, 2, 3, 4, 5]; // a=1, b=2, rest=[3,4,5]
const { name:n, age=21 } = student;      // rename + default

// Spread operator
const arr1 = [1, 2]; const arr2 = [3, 4];
const combined = [...arr1, ...arr2]; // [1,2,3,4]
const obj2 = { ...obj1, extra: "val" };

// Optional chaining ?. – no error if null/undefined
const city = user?.address?.city; // undefined (not error) if address is null

// Nullish coalescing ?? – default only for null/undefined
const theme = userPref ?? "light"; // "light" only if userPref is null/undefined
// vs || which triggers on ANY falsy (0, "", false too)

// Short-circuit for defaults
const name2 = inputName || "Guest"; // "Guest" if inputName is falsy
```

Part 2: PHP — Complete Study Notes

PHP (PHP: Hypertext Preprocessor) is a server-side scripting language. Unlike HTML/CSS/JS which run in the browser, PHP runs on the web server. The server processes PHP and sends only the resulting HTML to the browser — so users never see your PHP code.

Chapter 4.1 — PHP Syntax Fundamentals

```
<?php
// Single-line comment
# Also a single-line comment
/* Multi-line
   comment */

// Variables start with $ - loosely typed (no type declaration)
$name  = "Nitya";    // string
$age   = 21;         // integer
$cgpa  = 9.5;        // float
$active = true;      // boolean
$nothing = null;     // null

// echo outputs HTML
echo "Hello, $name!";           // variables expand in double quotes
echo "Hello, " . $name . "!";   // concatenation with dot
echo "CGPA: " . $cgpa . "<br>";
print "Also outputs, returns 1"; // alternative to echo
?>
```

NOTE: PHP variables inside double-quoted strings are automatically expanded. Single-quoted strings treat everything literally — no variable expansion.

Chapter 4.2 — Data Types & Type Functions

```
$str = "Hello";      gettype($str); // "string"
$int = 42;           gettype($int); // "integer"
$flt = 3.14;         gettype($flt); // "double" (PHP calls floats double)
$bool = true;        gettype($bool); // "boolean"
$arr  = [1, 2, 3];   gettype($arr); // "array"
>null = null;        gettype($null); // "NULL"

// Type casting
$str = "42";
(int)$str;    // 42      - explicit cast
(float)$str;  // 42.0
(bool)$str;   // true (non-empty string)
(string)42;   // "42"

// settype - changes variable type IN PLACE
settype($var, "integer"); // now $var is an int

// Type checking
is_string($var) // true/false
is_int($var)
```

```

is_float($var)
is_bool($var)
is_null($var)
is_array($var)
is_numeric($var) // true for "42" AND 42

```

Chapter 4.3 — Strings (Complete Reference)

```

$str = "Hello, World!";

strlen($str)           // 13 - character count
strtolower($str)       // "hello, world!"
strtoupper($str)       // "HELLO, WORLD!"
ucfirst($str)          // "Hello, world!" - capitalise first
ucwords("hello world") // "Hello World" - capitalise each word
trim("  hello  ")      // "hello" - strip whitespace both ends
ltrim("  hello")       // "hello" - strip left only
rtrim("hello  ")       // "hello" - strip right only

str_replace("World", "PHP", "Hello World") // "Hello PHP"
str_replace(["a", "e"], ["@", "3"], "age")  // "@g3"

strpos("Hello World", "World") // 6 (position) or false if not found
substr("Hello World", 6, 5)     // "World" (start, length)
str_repeat("ab", 3)             // "ababab"
str_pad("5", 3, "0", STR_PAD_LEFT) // "005"

explode(",", "a,b,c") // ["a","b","c"] - split string to array
implode(" | ", $arr)  // "a | b | c" - join array to string

sprintf("Name: %s, Score: %d%%", $name, $score) // formatted string
number_format(12345.678, 2) // "12,345.68"

htmlspecialchars("<script>alert(1)</script>")
// "&lt;script&gt;alert(1)&lt;/script&gt;" - safe for output
htmlspecialchars_decode("&lt;b&gt;") // "<b>" - reverse
strip_tags("<p>Hello <b>World</b></p>") // "Hello World"

```

Chapter 4.4 — Arrays (Indexed, Associative, Multi-dimensional)

```

// Indexed array
$fruits = ["apple", "banana", "cherry"];
$fruits[] = "date"; // append
$fruits[0]; // "apple"
count($fruits); // 4

// Associative array (key => value)
$student = [
    "name" => "Nitya",
    "age" => 21,
    "cgpa" => 9.5
];
$student["name"]; // "Nitya"
$student["email"] = "n@x.com"; // add new key

// Multidimensional array

```

```

$class = [
    ["name"=>"Nitya", "cgpa"=>9.5],
    ["name"=>"Shubham","cgpa"=>8.9]
];
$class[0]["name"];    // "Nitya"

// Array functions
in_array("banana", $fruits)    // true
array_key_exists("name", $student) // true
array_keys($student)           // ["name","age","cgpa"]
array_values($student)         // ["Nitya",21,9.5]
array_push($arr, "val")        // append
array_pop($arr)                // remove last
array_shift($arr)              // remove first
array_unshift($arr, "first")   // add to beginning
array_merge($arr1, $arr2)      // combine two arrays
array_slice($arr, 1, 2)        // elements index 1 and 2
sort($arr)                     // sort ascending (modifies in place)
rsort($arr)                    // sort descending
asort($assoc)                  // sort by value, keep keys
ksort($assoc)                  // sort by key
array_unique($arr)             // remove duplicates
array_reverse($arr)            // reverse order
array_search("banana", $arr)   // returns key/index or false

```

Chapter 4.5 — Control Flow

```

// if / elseif / else
if ($score >= 90)      echo "A";
elseif ($score >= 75)  echo "B";
elseif ($score >= 60)  echo "C";
else                   echo "Fail";

// switch
switch ($day) {
    case "Mon": echo "Monday"; break;
    case "Tue": echo "Tuesday"; break;
    default:    echo "Other day";
}

// Ternary and null coalescing
$label = ($score >= 50) ? "Pass" : "Fail";
$name  = $_POST["name"] ?? "Guest";    // ?? - if left is null/not set

// match expression (PHP 8+) - strict, no fall-through
$result = match($status) {
    "active"    => "User is active",
    "inactive"  => "User is inactive",
    default     => "Unknown status"
};

```

Chapter 4.6 — Loops

```

// for
for ($i = 1; $i <= 5; $i++) {
    echo $i . " ";    // 1 2 3 4 5
}

```

```
}

// while
$n = 10;
while ($n > 0) {
    echo $n . " ";
    $n -= 2;           // 10 8 6 4 2
}

// do-while (executes at least once)
do {
    echo "runs at least once";
    $n++;
} while ($n < 0);

// foreach - indexed array
foreach ($fruits as $fruit) {
    echo $fruit . "<br>";
}

// foreach - associative array (key => value)
foreach ($student as $key => $value) {
    echo "$key: $value <br>";
}

// break and continue
for ($i=0; $i<10; $i++) {
    if ($i == 3) continue; // skip 3
    if ($i == 7) break;    // stop at 7
    echo $i;              // 0 1 2 4 5 6
}
```

Chapter 4.7 — Functions

```
// Basic function
function add($a, $b) {
    return $a + $b;
}
echo add(3, 4); // 7

// Default parameters
function greet($name, $greeting = "Hello") {
    return "$greeting, $name!";
}
greet("Nitya"); // "Hello, Nitya!"
greet("Nitya", "Hi"); // "Hi, Nitya!"

// Pass by reference (&)
function double(&$val) { $val *= 2; }
$x = 5;
double($x);
echo $x; // 10 - original was changed

// Return multiple values (using array)
function minMax($arr) {
    return [min($arr), max($arr)];
}
```

```
[$min, $max] = minMax([3, 1, 4, 1, 5]);

// Variable functions and anonymous
$square = function($n) { return $n * $n; };
echo $square(5); // 25
```

Chapter 4.8 — Superglobals Reference

\$_GET	URL query parameters: ?name=Nitya&age=21 → \$_GET['name'] = 'Nitya'
\$_POST	Form data sent with method='post' — not visible in URL
\$_REQUEST	Combines \$_GET, \$_POST, \$_COOKIE — avoid (ambiguous source)
\$_SESSION	Server-side session storage — persists across page loads for one user
\$_COOKIE	Browser cookies sent with every request
\$_FILES	Uploaded file info: name, type, size, tmp_name, error
\$_SERVER	Server/request info: PHP_SELF, REQUEST_METHOD, HTTP_HOST, REMOTE_ADDR
\$_ENV	Environment variables (set on the server)
\$GLOBALS	All global variables accessible from anywhere

Chapter 4.9 — Date & Time Functions

```
date("Y-m-d") // "2026-05-05" - current date
date("d/m/Y") // "05/05/2026"
date("H:i:s") // "14:30:00" - current time
date("D, d M Y") // "Tue, 05 May 2026"
date("l") // "Tuesday" - full weekday name
date("N") // "2" - weekday number (1=Mon, 7=Sun)

time() // current Unix timestamp (seconds since 1970)
time() + 86400 // timestamp for tomorrow

// Create timestamp for specific date
mktime(0, 0, 0, 12, 25, 2026) // Christmas 2026
strtotime("next Monday") // timestamp for next Monday
strtotime("+3 days") // 3 days from now
strtotime("2026-12-25") // parse date string to timestamp

// Format a specific timestamp
date("d M Y", strtotime("2026-12-25")) // "25 Dec 2026"
```

Chapter 4.10 — Regular Expressions in PHP

```
// preg_match - returns 1 if match found, 0 if not
preg_match("/^[0-9]{10}$/", $phone); // exact 10 digits
preg_match("/^[A-Z][a-z]+$/", $name); // starts uppercase, rest lower
```

```
// Capture groups
preg_match("/([0-9]{4})-([0-9]{2})-([0-9]{2})/", "2026-05-05", $matches);
// $matches[0]="2026-05-05", $matches[1]="2026", $matches[2]="05", $matches[3]="05"

// preg_replace - replace pattern
preg_replace("/^[^0-9]/", "", $phone) // remove all non-digits
preg_replace("/\s+/", " ", $str) // collapse multiple spaces

// preg_match_all - find all matches
preg_match_all("/\d+/", "I have 3 cats and 12 dogs", $m);
// $m[0] = ["3", "12"]
```

Chapter 4.11 — Error Handling

```
// try-catch
try {
    if ($b == 0) throw new Exception("Division by zero");
    $result = $a / $b;
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
    echo " in " . $e->getFile() . " line " . $e->getLine();
} finally {
    echo "Cleanup always runs";
}

// Error reporting (in development)
error_reporting(E_ALL);
ini_set("display_errors", 1);

// Custom error handler
set_error_handler(function($errno, $errstr) {
    echo "Error [$errno]: $errstr";
});
```

Chapter 4.12 — Prepared Statements (SQL Injection Prevention)

Never put user input directly in SQL strings — use prepared statements to prevent SQL injection attacks.

```
// DANGEROUS - vulnerable to SQL injection
$name = $_POST["name"];
$conn->query("SELECT * FROM users WHERE name = '$name'"); // DON'T DO THIS

// SAFE - prepared statement with bound parameter
$stmt = $conn->prepare("SELECT * FROM users WHERE name = ?");
$stmt->bind_param("s", $name); // "s" = string, "i" = int, "d" = double
$stmt->execute();
$result = $stmt->get_result();
while ($row = $result->fetch_assoc()) {
    echo $row["name"];
}
$stmt->close();

// INSERT with prepared statement
$stmt = $conn->prepare("INSERT INTO students (name, email) VALUES (?, ?)");
$stmt->bind_param("ss", $name, $email);
```

```
$stmt->execute();  
echo "Inserted ID: " . $conn->insert_id;
```

Chapter 4.13 — include & require

```
// include - inserts another PHP file; continues even if file missing (warning)  
include "header.php";  
include "config.php";  
  
// require - inserts another PHP file; STOPS (fatal error) if file missing  
require "database.php"; // use for critical files  
require "functions.php";  
  
// _once variants - prevents double-loading  
include_once "header.php";  
require_once "functions.php"; // most common for function libraries
```

Part 3: Quick Reference Tables

HTML Tags Cheat Sheet

Tag	Purpose	Key Attributes
<code><a></code>	Hyperlink	<code>href</code> , <code>target</code> , <code>rel</code>
<code></code>	Image	<code>src</code> , <code>alt</code> , <code>width</code> , <code>height</code> , <code>align</code> , <code>border</code> , <code>usemap</code>
<code><table></code>	Table container	<code>border</code> , <code>cellpadding</code> , <code>cellspacing</code> , <code>width</code> , <code>align</code>
<code><tr></code>	Table row	<code>bgcolor</code>
<code><td></code> / <code><th></code>	Cell / Header cell	<code>colspan</code> , <code>rowspan</code> , <code>align</code> , <code>valign</code> , <code>bgcolor</code>
<code><form></code>	Form container	<code>action</code> , <code>method</code> , <code>enctype</code>
<code><input></code>	Form control	<code>type</code> , <code>name</code> , <code>value</code> , <code>required</code> , <code>pattern</code> , <code>min</code> , <code>max</code> , <code>placeholder</code>
<code><select></code>	Dropdown	<code>name</code> , <code>required</code> , <code>multiple</code> , <code>size</code>
<code><textarea></code>	Multi-line input	<code>name</code> , <code>rows</code> , <code>cols</code> , <code>minlength</code> , <code>maxlength</code>
<code><map></code> + <code><area></code>	Image map	<code>name (map)</code> ; <code>shape</code> , <code>coords</code> , <code>href</code> , <code>alt (area)</code>
<code><frameset></code>	Frame layout	<code>rows</code> , <code>cols</code>
<code><frame></code>	Individual frame	<code>src</code> , <code>name</code> , <code>noresize</code>
<code><marquee></code>	Scrolling text	<code>behavior</code> , <code>direction</code> , <code>scrollspeed</code>
<code>//</code>	Lists	<code>type</code> , <code>start (ol)</code>
<code><dl>/<dt>/<dd></code>	Definition list	–
<code><div></code>	Generic block container	<code>id</code> , <code>class</code> , <code>style</code>
<code></code>	Generic inline container	<code>id</code> , <code>class</code> , <code>style</code>
<code><meta></code>	Metadata	<code>charset</code> , <code>name</code> , <code>content</code> , <code>viewport</code>
<code><link></code>	External resource	<code>rel</code> , <code>href</code> , <code>type</code>
<code><script></code>	JavaScript	<code>src</code> , <code>type</code> , <code>defer</code> , <code>async</code>

PHP Functions Cheat Sheet

Function	What it does
<code>echo</code> / <code>print</code>	Output text/HTML to the browser
<code>\$_POST['key']</code>	Read form data submitted via POST method

<code>\$_GET['key']</code>	Read URL query parameter
<code>\$_FILES['key']</code>	Access uploaded file info (name, type, size, tmp_name, error)
<code>\$_SESSION['key']</code>	Server-side session variable (needs session_start() first)
<code>\$_COOKIE['key']</code>	Read browser cookie
<code>\$_SERVER['REQUEST_METHOD']</code>	'GET' or 'POST' — detect how page was accessed
<code>isset(\$var)</code>	true if variable exists AND is not null
<code>empty(\$var)</code>	true if variable is "", 0, false, null, [], or not set
<code>unset(\$var)</code>	Destroy a variable
<code>gettype(\$var)</code>	Returns type as string: 'string','integer','boolean', etc.
<code>settype(\$var, 'type')</code>	Convert variable type in-place
<code>htmlspecialchars(\$str)</code>	Encode < > & " ' for safe HTML output (XSS prevention)
<code>header('Location: url')</code>	Redirect browser to another URL (before any HTML)
<code>session_start()</code>	Begin/resume session — must be very first line
<code>session_destroy()</code>	End the session and delete all data
<code>setcookie(name, val, expiry, path)</code>	Set a browser cookie
<code>file_put_contents(file, data, FLAG)</code>	Write/append to a file
<code>file(path, FLAGS)</code>	Read file into an array of lines
<code>file_exists(path)</code>	true if file or directory exists
<code>move_uploaded_file(tmp, dest)</code>	Move uploaded file from temp to permanent location
<code>explode(delim, str)</code>	Split string into array by delimiter
<code>implode(glue, arr)</code>	Join array elements into a string
<code>str_replace(search, replace, str)</code>	Replace all occurrences in string
<code>strlen(\$str)</code>	Character count of string
<code>trim(\$str)</code>	Remove whitespace from both ends
<code>date('format')</code>	Format current date/time
<code>time()</code>	Current Unix timestamp (seconds since 1970-01-01)
<code>count(\$arr)</code>	Number of elements in array
<code>in_array(val, arr)</code>	true if value exists in array
<code>array_push(\$arr, val)</code>	Append value to end of array
<code>sort(\$arr) / rsort(\$arr)</code>	Sort array ascending / descending
<code>preg_match('/pattern/', \$str)</code>	Returns 1 if regex matches, 0 if not
<code>new mysqli(host, user, pass, db)</code>	Connect to MySQL database
<code>\$conn->query(\$sql)</code>	Execute SQL query, returns result object
<code>\$res->fetch_assoc()</code>	Fetch next row as associative array

<code>\$conn->prepare(\$sql)</code>	Prepare parameterised SQL (prevents injection)
<code>print_r(\$var)</code>	Human-readable dump of variable (debug)
<code>var_dump(\$var)</code>	Detailed dump with types (debug)

JavaScript Methods Cheat Sheet

Method / Property	What it does
<code>document.getElementById(id)</code>	Select element by its id attribute
<code>document.querySelector(css)</code>	Select first element matching CSS selector
<code>document.querySelectorAll(css)</code>	Select ALL matching elements (NodeList)
<code>element.textContent = 'text'</code>	Set/read plain text content
<code>element.innerHTML = 'x'</code>	Set/read HTML content
<code>element.value</code>	Read/write form input value
<code>element.style.property = val</code>	Set inline CSS style
<code>element.classList.add/remove/toggle</code>	Manage CSS classes on element
<code>element.setAttribute(name, val)</code>	Set any HTML attribute
<code>element.addEventListener(evt, fn)</code>	Attach event handler
<code>document.createElement(tag)</code>	Create a new DOM element
<code>parent.appendChild(child)</code>	Add child element at end of parent
<code>element.remove()</code>	Remove element from DOM
<code>alert(msg)</code>	Show browser popup with OK button
<code>confirm(msg)</code>	Show popup with OK/Cancel, returns boolean
<code>prompt(msg)</code>	Show input dialog, returns string or null
<code>window.open(url, target, features)</code>	Open new browser window/tab
<code>win.document.write(html)</code>	Write HTML to a new window
<code>window.print()</code>	Open browser print dialog
<code>setTimeout(fn, ms)</code>	Run fn once after ms milliseconds
<code>setInterval(fn, ms)</code>	Run fn every ms milliseconds
<code>clearInterval(id)</code>	Stop a repeating setInterval
<code>eval(str)</code>	Execute string as JavaScript code (use with caution)
<code>JSON.stringify(obj)</code>	Convert object to JSON string
<code>JSON.parse(str)</code>	Convert JSON string to object
<code>Array.push/pop/shift/unshift</code>	Add/remove elements from array ends
<code>arr.map(fn)</code>	Return new array transforming each element
<code>arr.filter(fn)</code>	Return new array with elements passing test

<code>arr.reduce(fn, init)</code>	Reduce array to single value
<code>arr.find(fn)</code>	Return first element passing test
<code>arr.indexOf(val)</code>	Return index of value (-1 if not found)
<code>arr.includes(val)</code>	Return true if value exists in array
<code>arr.join(sep)</code>	Join array to string with separator
<code>arr.sort((a,b)=>a-b)</code>	Sort numerically ascending
<code>str.split(sep)</code>	Split string to array
<code>str.trim()</code>	Remove whitespace from both ends
<code>str.toUpperCase() / toLowerCase()</code>	Change case
<code>str.includes(substr)</code>	true if substring found
<code>str.replace(pattern, replacement)</code>	Replace first/all occurrences
<code>Math.round/floor/ceil(n)</code>	Round number
<code>Math.max(...arr) / Math.min(...arr)</code>	Max/min of values
<code>Math.random()</code>	Random float 0 to <1
<code>parseInt(str) / parseFloat(str)</code>	Parse string to number
<code>isNaN(val)</code>	true if value is Not a Number